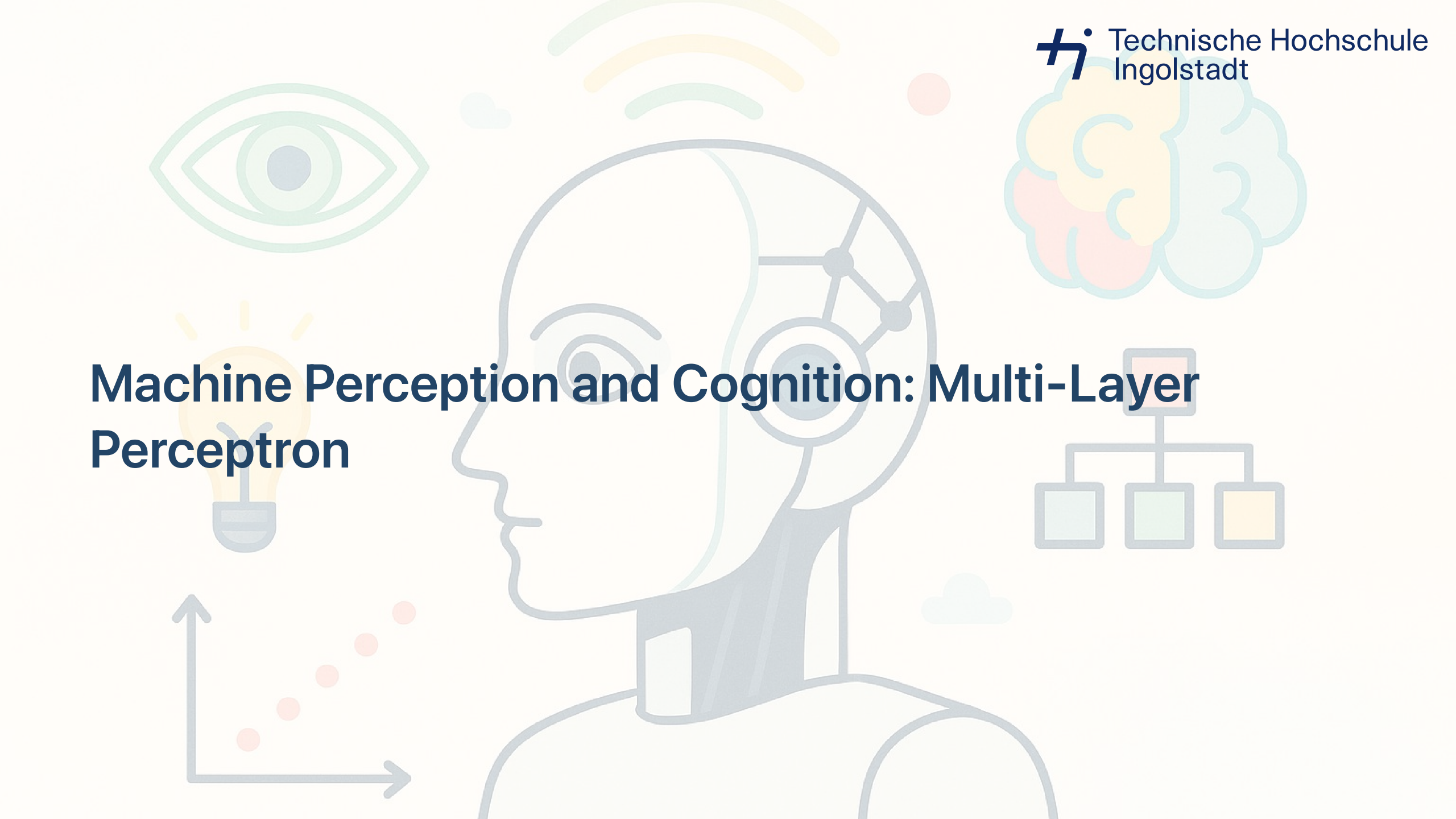
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Multi-Layer Perceptron

Feature vector and weights

In the Perceptron, we had the augmented feature vector:

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_n & 1 \end{bmatrix}^\top$$

And a hyperplane defined by a weight vector of the same dimensionality:

$$\mathbf{w} = \begin{bmatrix} w_1 & w_2 & \cdots & w_n & w_{n+1} \end{bmatrix}^\top$$

Binary classification in a Perceptron

If $\mathbf{w}$ and $\mathbf{x}$ are orthogonal to each other, then $\mathbf{w}^\top \mathbf{x} = 0$

If $\mathbf{w}$ and $\mathbf{x}$ are codirectional, then $\mathbf{w}^\top \mathbf{x} = ||\mathbf{w}|| \cdot ||\mathbf{x}||$

If $\mathbf{w}$ and $\mathbf{x}$ are opposite, then $\mathbf{w}^\top \mathbf{x} = -||\mathbf{w}|| \cdot ||\mathbf{x}||$

Scalar product for decision boundary

The sign of the scalar product tells us on which side of the decision boundary point $\mathbf{x}$ lies:

$$\mathbf{w}^\top \mathbf{x} = \sum_i w_i \cdot x_i$$

- If $sign(\mathbf{w}^\top \mathbf{x}) = 0$, then $\mathbf{x}$ is on the hyperplane.
- If $sign(\mathbf{w}^\top \mathbf{x}) = +1$, then $\mathbf{x}$ is "above" the hyperplane.
- If $sign(\mathbf{w}^\top \mathbf{x}) = -1$, then $\mathbf{x}$ is "below" the hyperplane.

XOR problem

Can we solve the XOR function with this?

x_1	x_2	$XOR(x_1, x_2)$
-1	-1	-1
-1	+1	+1
+1	-1	+1
+1	+1	-1

XOR problem

One solution! Feature engineering!

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_1 \cdot x_2 \end{bmatrix}^\top$$

XOR problem

x_1	x_2	$x_1 \cdot x_2$	$XOR(x_1, x_2)$
-1	-1	+1	-1
-1	+1	-1	+1
+1	-1	-1	+1
+1	+1	+1	-1



XOR problem

But feature engineering gets very complex very fast...

Image source: https://www.pexels.com/photo/the-large-hadron-collider-at-geneva-switzerland-6855582/

From Perceptron to multi-layer perceptron

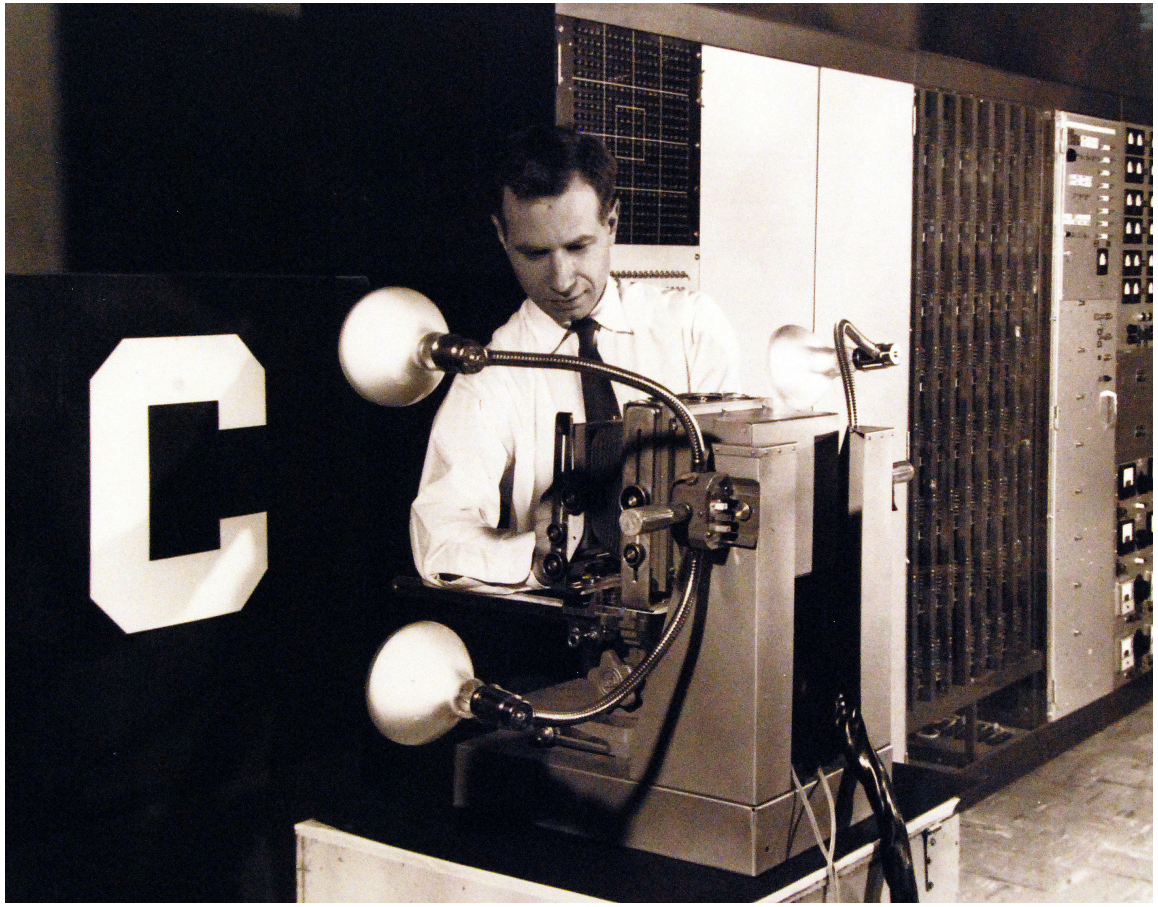


Image source: National Museum of the U.S. Navy, Public domain, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:330-PSA-80-60_(USN_710739)_(20897323365).jpg

Perceptron

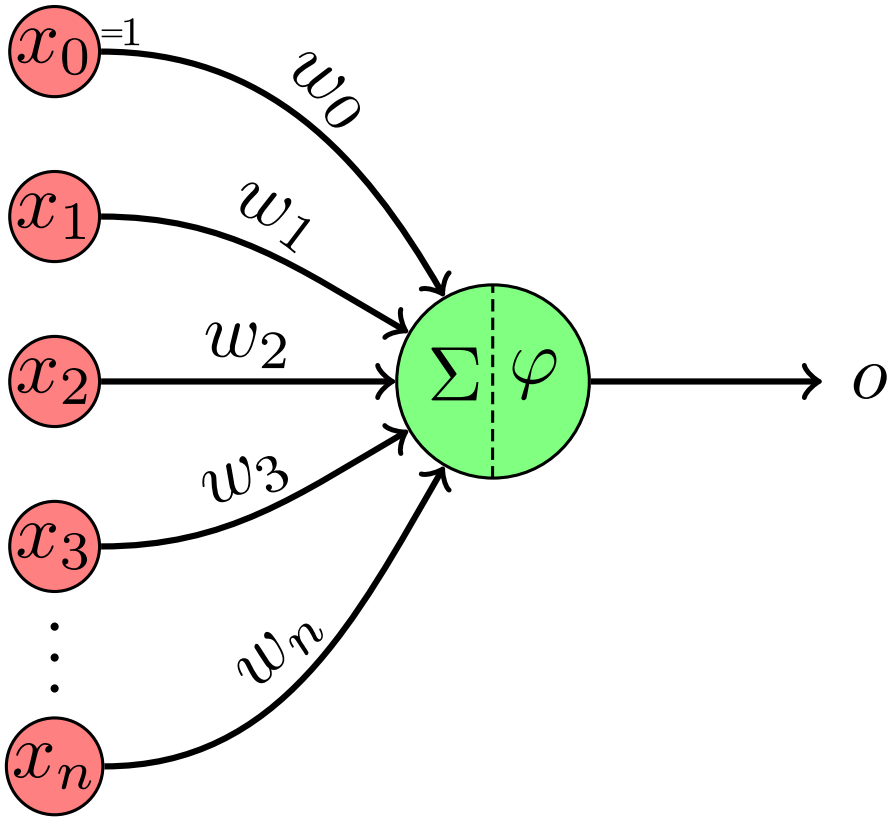


Image source: MartinThoma, CC0, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Perceptron-unit.svg

Multi-layer perceptron

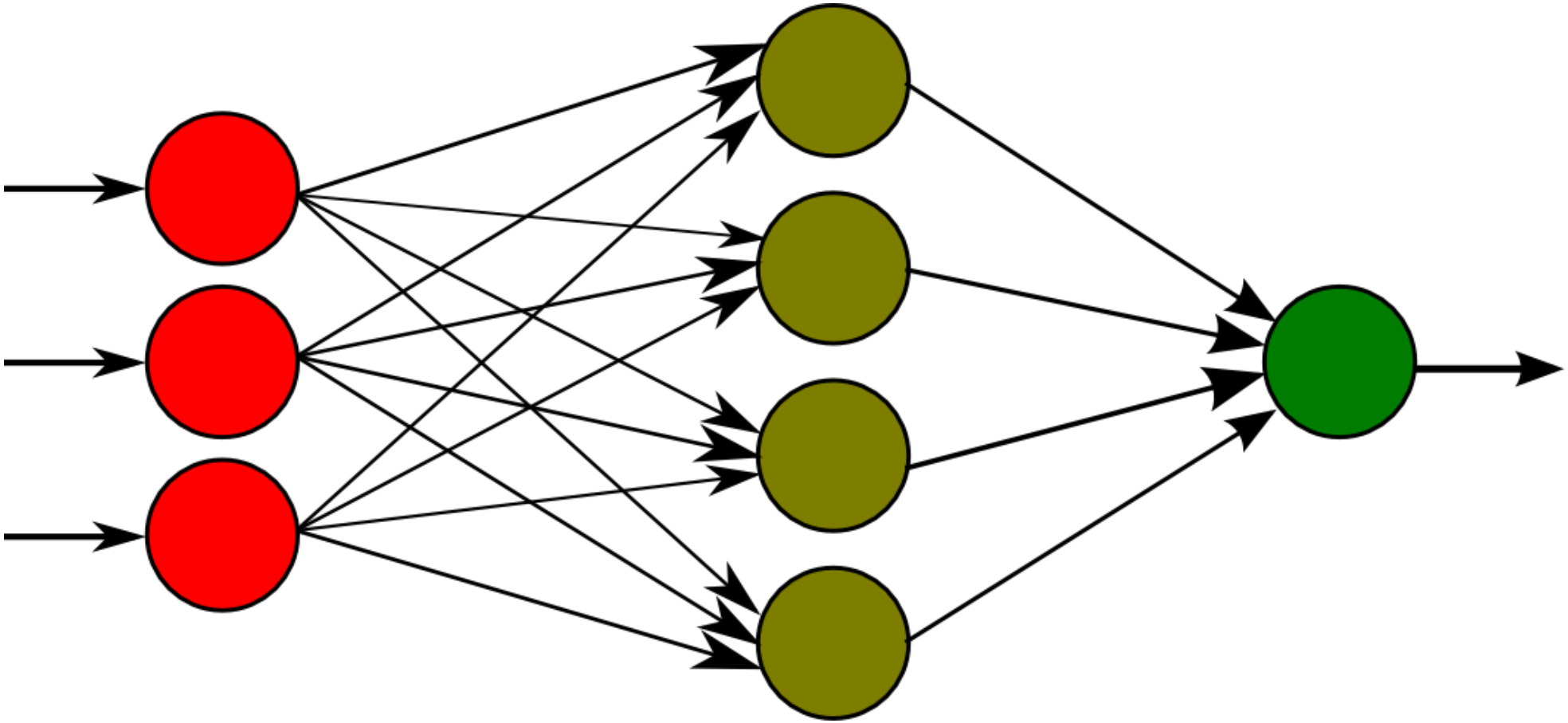


Image source: Sky99, CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:MultiLayerPerceptron.png

Limitations of the single Perceptron

A single Perceptron defines **one linear hyperplane** — it can only classify **linearly separable** data.

Real-world problems are rarely linearly separable:

- 🐱 vs. 🐶 in images
- 🎵 genre from audio features
- 🗣️ speech recognition from spectrograms

The XOR example already showed: a single hyperplane **cannot** separate some simple patterns.

Feature engineering as a workaround

Feature engineering transforms the raw features into a new space where linear separation becomes possible:

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1 & x_2 & x_1 \cdot x_2 & x_1^2 & x_2^2 & \cdots \end{bmatrix}^\top$$

Problems with feature engineering:

- 🔨 Requires deep domain expertise
- 📈 Number of features grows exponentially with the polynomial degree
- 🔄 Must be redesigned for every new problem
- 🚫 Does not scale to high-dimensional data (images, audio, text)

Why non-linear models?

Instead of manually crafting features, we want a model that **learns the right representation automatically** from data.

Key insight: composing multiple simple linear transformations with **non-linear functions** in between creates a powerful non-linear model:

Why non-linear models?

$$f(\mathbf{x}) = \sigma\left(\mathbf{W}^{(2)} \begin{bmatrix} \sigma\left(\mathbf{W}^{(1)}\mathbf{x}\right) \\ 1 \end{bmatrix}\right)$$

where the last entry of $\mathbf{x}$ is 1 and the bias is folded into each $\mathbf{W}^{(l)}$ — no separate bias term needed. Function σ is a non-linear activation function.

- No manual feature design needed.
- The model discovers its own internal representation.
- The same architecture works for images, audio, text, ...

The artificial neuron

A single artificial neuron computes a **weighted sum** of its inputs and passes the result through an **activation function** σ.

Using input $\mathbf{x}$ (with 1 as its last entry) and weight vector $\mathbf{w}$ (with $w_{n+1} = b$), the bias needs no separate term:

$$z = \mathbf{w}^\top \mathbf{x} = \sum_{i=1}^{n} w_i x_i + \underbrace{w_{n+1}}_{=b} \cdot 1 = \sum_{i=1}^{n} w_i x_i + b$$

$$a = \sigma(z)$$

Why do we need activation functions?

Without a non-linear activation function, stacking multiple layers is equivalent to a single linear transformation:

$$\mathbf{W}^{(2)}\left(\mathbf{W}^{(1)}\mathbf{x}+\mathbf{b}^{(1)}\right)+\mathbf{b}^{(2)}=\underbrace{\mathbf{W}^{(2)}\mathbf{W}^{(1)}}_{\mathbf{W}'}\mathbf{x}+\underbrace{\mathbf{W}^{(2)}\mathbf{b}^{(1)}+\mathbf{b}^{(2)}}_{\mathbf{b}'}$$

(The two layers collapse into a single weight matrix $\mathbf{W}'$ and bias $\mathbf{b}'$ — the composition of two linear maps is still linear.)

➡️ Still just a linear model! Non-linear activations **break this collapse** and let the network represent complex functions.

Common activation functions

Name	Formula	Range
Sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}}$	$(0, 1)$
Tanh	$\tanh(z) = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1, 1)$
ReLU	$\mathrm{ReLU}(z) = \max(0, z)$	$[0, \infty)$

❗ We will use σ as the non-linear activation function as notation! ❗

Sigmoid

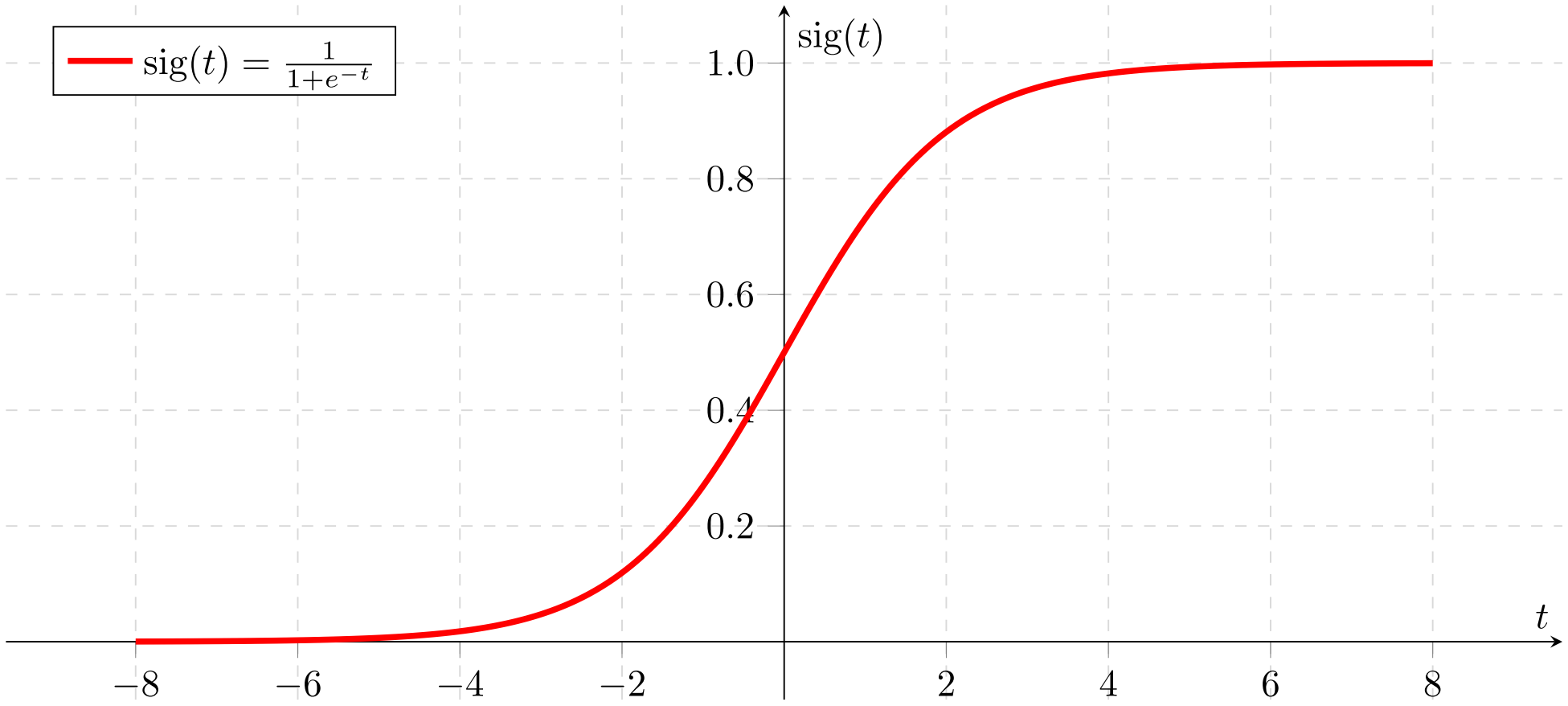


Image source: MartinThoma, CC0, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Sigmoid-function-2.svg

Tanh

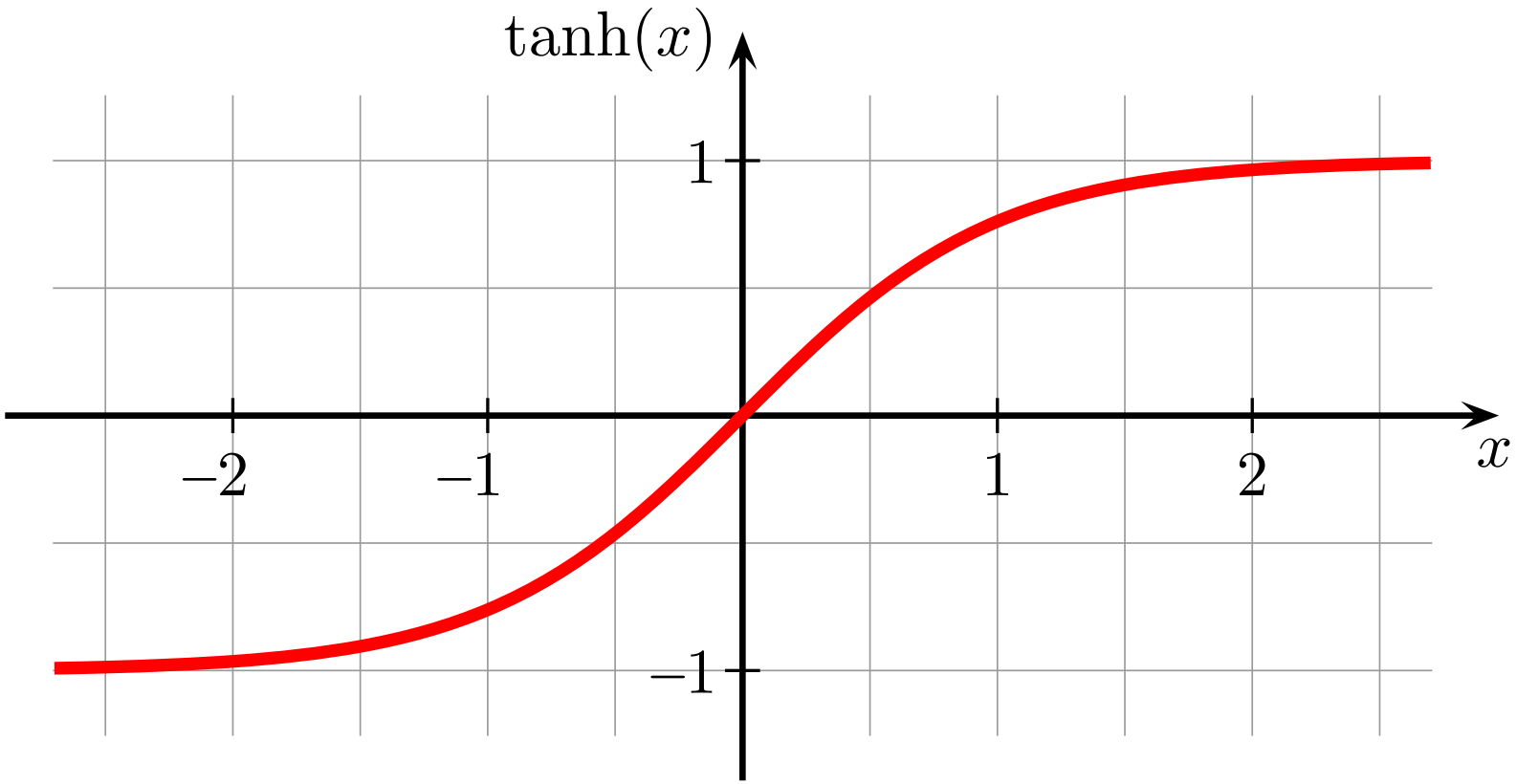


Image source: Geek3, CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Hyperbolic_Tangent.svg

ReLU

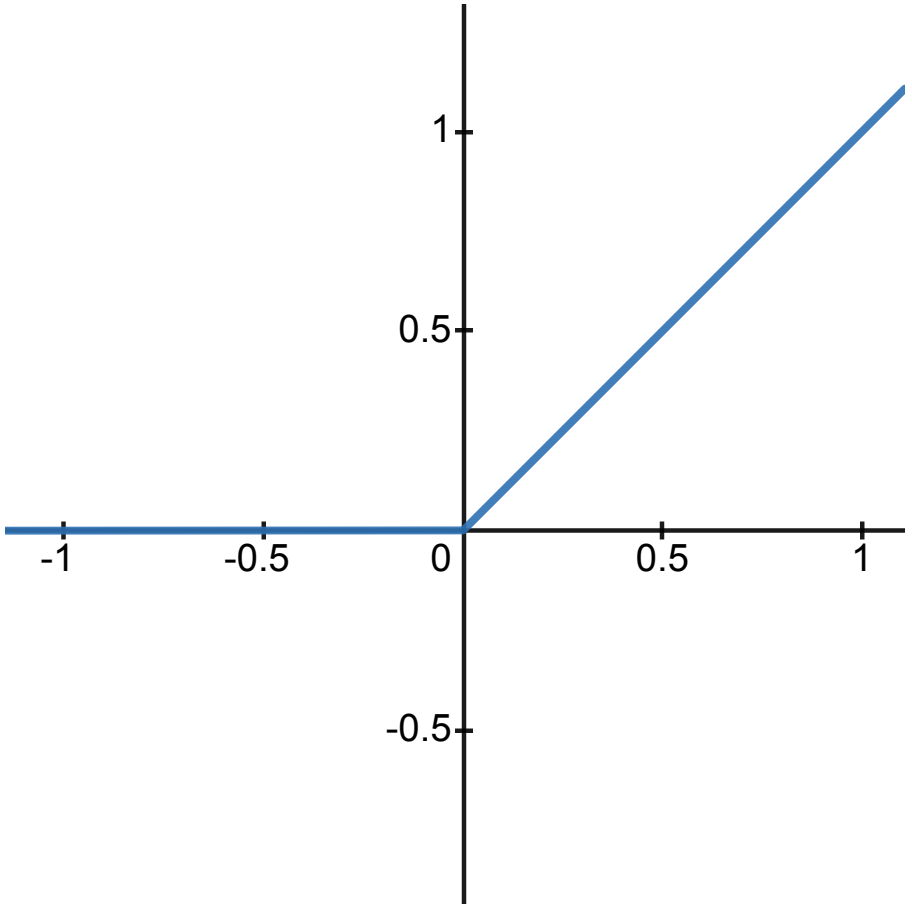


Image source: EthanChadwickBuckner, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:ReLU_Activation_Function_Plot.svg

Activation function properties

Sigmoid: outputs are always positive and saturate near 0 and 1

Tanh: zero-centred, but still saturates

ReLU: no saturation for $z > 0$; sparse activations (many neurons output 0)

➡️ **ReLU** and its variants are the default choice for hidden layers in modern networks.

MLP architecture

An MLP is a sequence of **fully connected layers**. Each layer l consists of n_l neurons:

$$\underbrace{\mathbf{x}}_{\text{input}} \longrightarrow \underbrace{\text{hidden layer } 1}_{n_1 \text{ neurons}} \longrightarrow \cdots \longrightarrow \underbrace{\text{hidden layer } L-1}_{n_{L-1} \text{ neurons}} \longrightarrow \underbrace{\text{layer } L \text{ (output)}}_{n_L \text{ neurons}} \longrightarrow \underbrace{\hat{\mathbf{y}}}_{\text{output}}$$

- **Input layer**: passes the raw feature vector $\mathbf{x} \in \mathbb{R}^{n_0}$
- **Hidden layers**: intermediate learned representations
- **Output layer**: produces predictions $\hat{\mathbf{y}}$

Every neuron in layer l is connected to **every** neuron in layer $l-1$ — hence "fully connected".

From weight vector to weight matrix

In the Perceptron, a **single output neuron** uses one weight vector $\mathbf{w} \in \mathbb{R}^{n+1}$:

$$z = \mathbf{w}^\top \mathbf{x}$$

In a hidden layer of an MLP, we need m outputs — one per neuron. Each neuron j has its own weight vector, forming the j-th row of the weight matrix:

$$z_j = \mathbf{w}_j^\top \mathbf{x}, \quad j = 1, \ldots, m$$

Stacking all m rows gives the **weight matrix** $\mathbf{W} \in \mathbb{R}^{m \times (n+1)}$:

$$\mathbf{z} = \mathbf{W}\,\mathbf{x}$$

All m outputs are computed in a single matrix–vector multiplication.

Bias in the weight matrix

Just as in the Perceptron we folded the bias into the feature vector, in the MLP we fold the **bias vector $\mathbf{b}^{(l)}$** into the **weight matrix $\mathbf{W}^{(l)}$** as its last column:

$$\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times (n_{l-1}+1)}, \quad \text{last column} = \mathbf{b}^{(l)}$$

The activation vector is augmented with a constant 1:

$$\mathbf{a}^{(l-1)} = \begin{bmatrix} a_1^{(l-1)} & \cdots & a_{n_{l-1}}^{(l-1)} & 1 \end{bmatrix}^\top \in \mathbb{R}^{n_{l-1}+1}$$

The pre-activation is then a single matrix–vector product — **no separate bias term**:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\, \mathbf{a}^{(l-1)}$$

Layer notation

Each layer $l \in \{1, \ldots, L\}$ is described by a **weight matrix** $\mathbf{W}^{(l)}$ that includes the bias in its last column:

$$\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times (n_{l-1}+1)}, \quad \text{last column} = \mathbf{b}^{(l)}$$

The (j, k)-th entry $W_{jk}^{(l)}$ is the weight from neuron k in layer $l-1$ to neuron j in layer l; the last column $\mathbf{b}^{(l)}$ contains the biases.

The total number of learnable parameters in layer l is:

$$n_l \cdot n_{l-1} + n_l = n_l \left(n_{l-1} + 1\right)$$

Counting parameters — example

Consider an MLP with architecture $784 \to 256 \to 128 \to 10$ (e.g., for digit classification):

Layer	Inputs n_{l-1}	Neurons n_l	Parameters $n_l\,(n_{l-1}+1)$
1	784	256	$256 \times 785 = 200\,960$
2	256	128	$128 \times 257 = 32\,896$
3	128	10	$10 \times 129 = 1\,290$
Total			**235,146**

A modest three-layer MLP already needs **235k learnable parameters**. Each parameter must be learned — this is the job of a training algorithm, which we will cover later.

Forward pass — single layer

For layer l, the forward pass computes:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\,\mathbf{a}^{(l-1)}$$

$$\mathbf{a}^{(l)} = \sigma\left(\mathbf{z}^{(l)}\right)$$

where σ is applied **element-wise**.

- $\mathbf{a}^{(l-1)}$ — augmented activation from the previous layer (with appended 1)
- $\mathbf{z}^{(l)}$ — pre-activation vector of layer l
- $\mathbf{a}^{(l)}$ — activation vector (output) of layer l

Forward pass — full network

Applying the single-layer rule repeatedly gives the full forward pass through an L-layer MLP:

$$\mathbf{a}^{(0)} = \mathbf{x}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\,\mathbf{a}^{(l-1)}, \quad l = 1, \ldots, L$$

$$\mathbf{a}^{(l)} = \sigma\left(\mathbf{z}^{(l)}\right), \quad l = 1, \ldots, L$$

$$\hat{\mathbf{y}} = \mathbf{a}^{(L)}$$

Each $\mathbf{a}^{(l)}$ has a trailing 1 appended for the bias (as defined in the weight matrix slide).

The output of each layer becomes the input of the next — this is called **feed-forward** computation.

XOR solved with MLP

A single Perceptron cannot separate XOR. A two-layer MLP can — because **each hidden neuron learns one linear boundary** and the output layer combines them.

With inputs $x_1, x_2 \in \{-1, +1\}$ and the sign activation, two hidden neurons with example weights:

- **Neuron 1** (detects both $+1$): $z_1 = x_1 + x_2 - 1.5$
- **Neuron 2** (detects both -1): $z_2 = -x_1 - x_2 - 1.5$
- **Output** (fires when neither fires): $z_{\text{out}} = -a_1 - a_2 - 1$

The hidden neurons transform the inputs into a **linearly separable** representation.

➡️ This is precisely the **automatic feature engineering** that replaces the manual product $x_1 \cdot x_2$.

XOR solved with MLP — activation table

x_1	x_2	$a_1 = \text{sign}(z_1)$	$a_2 = \text{sign}(z_2)$	$\hat{y} = \text{sign}(z_{\text{out}})$	XOR
-1	-1	-1	$+1$	-1	-1 ✓
-1	$+1$	-1	-1	$+1$	$+1$ ✓
$+1$	-1	-1	-1	$+1$	$+1$ ✓
$+1$	$+1$	$+1$	-1	-1	-1 ✓

- **Neuron 1** fires only for $(+1, +1)$ — detects "both inputs positive"
- **Neuron 2** fires only for $(-1, -1)$ — detects "both inputs negative"
- Output fires $(+1)$ only when **neither** hidden neuron fires → inputs differ

Universal approximation

The **Universal Approximation Theorem** states that an MLP with:

- a **single hidden layer**
- a **sufficient number of neurons**
- a **non-linear activation function**

can approximate **any continuous function** to arbitrary precision.

➡️ MLPs are theoretically powerful enough to model any real-world mapping — but we still need to **learn** the right weights.

Pros of the multi-layer perceptron

- ✅ **Non-linear**: can learn arbitrary decision boundaries
- ✅ **Automatic feature learning**: no manual feature engineering
- ✅ **Universal approximation**: can represent any continuous function
- ✅ **General purpose**: same architecture for images, audio, text, ...
- ✅ **Scalable**: more neurons / layers → more capacity
- ✅ **Data-driven**: improves with more training data

Cons of the multi-layer perceptron

- ❌ **Black box**: learned representations are hard to interpret
- ❌ **Data hungry**: needs large amounts of labeled training data
- ❌ **Computationally expensive**: many parameters to optimize
- ❌ **Prone to overfitting**: memorizes noise if the network is too large
- ❌ **Many hyperparameters**: number of layers, neurons, activation functions, ...

Multi-layer perceptron

Recap!

- 🔲 A single Perceptron is limited to **linearly separable** problems.
- 🛠️ Feature engineering solves non-linear problems but is **manual and brittle**.
- 🔗 An MLP stacks fully connected layers with **non-linear activation functions**.
- ⚡ **ReLU** and its variants are a commonly used hidden-layer activation.
- ➡️ The **forward pass** propagates inputs layer by layer: $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)}$, $\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$.
- 🌐 The **Universal Approximation Theorem** guarantees that MLPs can represent any continuous function.
- ⚖️ MLPs are powerful but data-hungry, expensive, and hard to interpret.